50. Inference Serving: Batching, Queues, Sidecars

50. Inference Serving: Batching, Queues, Sidecars

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `50-inference-serving`

Learning objectives

- Queue inference requests
- Dynamic batch for GPU efficiency
- Use sidecar patterns for model runtimes
- Expose SLOs for latency vs throughput

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Serving shapes

- **Sync API**: simple, limited throughput
- **Queue + workers**: smooth spikes
- **Dynamic batching**: wait briefly to batch requests for GPU

Sidecar

Go app handles auth, quotas, routing; model runtime (Python/C++/TensorRT) runs beside it. Communicate via localhost gRPC/HTTP.

Tradeoffs

Batching raises latency p50 slightly to improve throughput — tune wait windows with metrics.

Common mistakes (especially from Python instincts)

- One goroutine per GPU call unbounded
- No max queue depth (OOM)
- Ignoring tail latency

Exercises

Work in `exercises/50-inference-serving/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/50-inference-serving/` until you have an honest failing attempt.

Self-check

1. Why dynamic batching?
2. What is a sidecar?
3. Queue depth purpose?

Further reading

- https://go.dev/blog/pipelines